

osmnx-sandbox

November 17, 2025

```
[2]: %reload_ext autoreload
      %autoreload 2
```

```
[3]: from kret_studies import *
      from kret_studies.notebook import *
      from kret_studies.complex import *

      logger = get_notebook_logger()
```

Loaded environment variables from /Users/Akseldkw/coding/Columbia/RL-Project/.env.

INFO:datasets:JAX version 0.7.2 available.

```
[4]: from waymo_agent import *
```

```
[5]: G = ox.load_graphml(filepath=MAP_DIR / "manhattan.graphml")
```

```
[6]: # fig, ax = ox.plot_graph(G, node_size=0, edge_linewidth=0.5)
```

```
[7]: # 1. Load graph (if not already loaded)
      # G = ox.load_graphml("manhattan.graphml")

      # 2. Define mappings for Colors AND Widths
      config = {
          "motorway": {"color": "#ff0000", "width": 3.0}, # Red, Thickest
          "motorway_link": {"color": "#ff0000", "width": 3.0},
          "trunk": {"color": "#ff6600", "width": 2.5}, # Orange, Thick
          "primary": {"color": "#ffac12", "width": 2.0}, # Gold, Medium-Thick
          "secondary": {"color": "#f4f458", "width": 1.0}, # Yellow, Medium
          "tertiary": {"color": "#ffffff", "width": 0.8}, # White, Thin
          "residential": {"color": "#555555", "width": 0.5}, # Grey, Thinnest
          "default": {"color": "#333333", "width": 0.5}, # Fallback
      }

      # 3. Generate the lists
      edge_colors = []
      edge_widths = []
```

```

for u, v, data in G.edges(data=True):
    # Get highway type (handle lists if necessary)
    highway_type = data.get("highway", "default")
    if isinstance(highway_type, list):
        highway_type = highway_type[0]

    # Fetch style from config, or use default if type not found
    style = config.get(highway_type, config["default"])

    edge_colors.append(style["color"])
    edge_widths.append(style["width"])

# 4. Plot with both attributes
fig, ax = ox.plot_graph(
    G,
    node_size=0,
    edge_color=edge_colors,
    edge_linewidth=edge_widths, # <--- Pass the new list here
    bgcolor="black",
    figsize=(12, 12), # Bigger size helps see the width differences
    show=True,
)

```



```
[8]: # fig.savefig(IMG_DIR / "manhattan_map.png", dpi=600, bbox_inches="tight",  
      ↪facecolor=fig.get_facecolor())
```

```
[9]: len(G.nodes), len(G.edges)
```

```
[9]: (4619, 9901)
```

0.1 Sparsify the graph

```
[10]: G = ox.load_graphml(filepath=MAP_DIR / "manhattan.graphml")
```

```
[11]: G_sparse = sparsify_graph(G)
```

Removing 4782 non-major edges...
Graph after edge removal: 3505 nodes
Cleaning edge attributes before simplification...
Simplifying topology...

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[11], line 1  
----> 1 G_sparse = sparsify_graph(G)  
  
File ~/coding/Columbia/RL-Project/waymo_agent/graph/sparsify.py:66, in  
    ↪sparsify_graph(G)  
    64 print("Simplifying topology...")  
    65 G_sparsified.graph["simplified"] = False  
--> 66 G_simplified = ox.simplify_graph(G_sparsified)  
    68 assert isinstance(G_simplified, nx.MultiDiGraph)  
    69 print(f"Final simplified graph: {G_simplified.number_of_nodes()} nodes"  
  
File ~/micromamba/envs/kret_312/lib/python3.12/site-packages/osmnx/  
    ↪simplification.py:406, in simplify_graph(G, node_attrs_include,  
    ↪edge_attrs_differ, remove_rings, track_merged, edge_attr_aggs)  
    404     agg_func = edge_attr_aggs[attr_name]  
    405     path_attributes[attr_name] = agg_func(attr_values)  
--> 406 elif len(set(attr_values)) == 1:  
    407     # if there's only 1 unique value, keep that single value  
    408     path_attributes[attr_name] = attr_values[0]  
    409 else:  
    410     # otherwise, if there are multiple uniques, keep one of each  
  
TypeError: unhashable type: 'list'
```

[]: